Rekursion – Fortgeschrittene Themen

Einführung in die Programmierung 1

TU Wien, Wintersemester 25

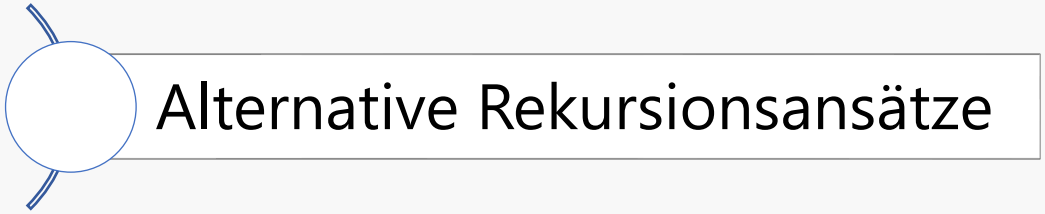
Was wurde zuletzt besprochen?

- Rekursion allgemein
- Lineare Rekursion
- Endrekursion

Was wird in diesem Foliensatz besprochen?

- Verzweigte Rekursion
- Wechselseitige Rekursion
- Verschachtelte Rekursion

Was lernen Sie kennen?



Verzweigte Rekursion

Jeder nichtterminierende rekursive Aufruf erzeugt zwei oder mehrere Aufrufe

Es entsteht ein Aufrufbaum

Fibonacci-Zahlen

$$\text{fibonacci(n)} = \begin{cases} 1 & falls\ n\ = 1\ oder\ n = 2 \\ \text{fibonacci(n} - 1) + \text{fibonacci(n} - 2) & sonst\ (n > 2) \end{cases}$$

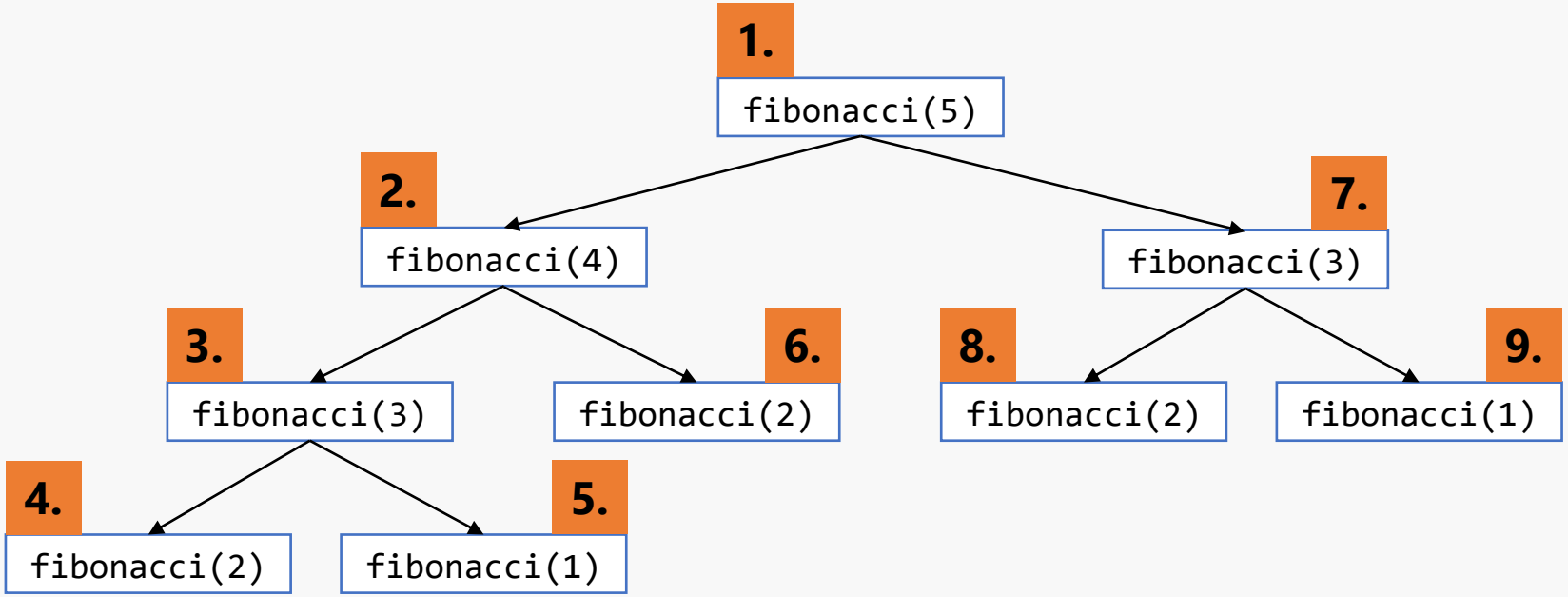
Zwei Aufrufe pro Rekursionsschritt

Beispiel (Fibonacci-Zahlen)

```java
public class FibonacciTest {

    // n >= 1
    private static long fibonacci(int n) {
        if (n <= 2) {
            return 1;
        } else {
            return fibonacci(n - 1) + fibonacci(n - 2);
        }
    }

    public static void main(String[] args) {
        System.out.println(fibonacci(1));
        System.out.println(fibonacci(3));
        System.out.println(fibonacci(10));
    }

}
```

```
1
2
55
```

Aufrufbaum für fibonacci (5)



Anzahl der Aufrufe wächst schnell an!

Beispiel (Binomialkoeffizient)

$$\mathrm{binom}(\mathrm{n}, \mathrm{k}) = \begin{cases} 1 & falls\ k\ = 0\ oder\ n = k \\ \mathrm{binom}(\mathrm{n} - 1, k\ - 1) + \mathrm{binom}(\mathrm{n}\ - 1, \mathrm{k}) & sonst \end{cases}$$

```
// 0 <= k <= n
private static long binomialCoefficient(int n, int k) {
    if (k == 0 || n == k) {
        return 1;
    }
    return binomialCoefficient(n - 1, k - 1) + binomialCoefficient(n - 1, k);
}
```

Türme von Hanoi

Ausgangssituation

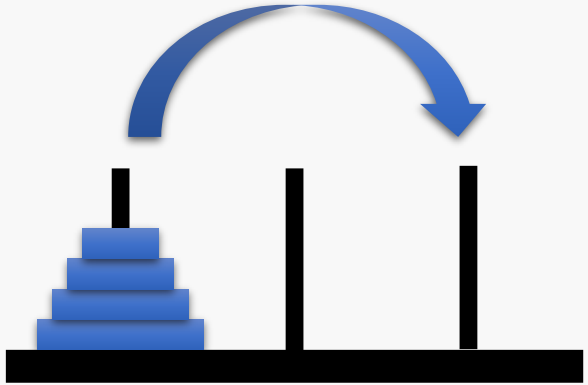
- Turm mit n Scheiben auf dem linken Stab von drei Stäben
- Jede Scheibe liegt auf einer größeren Scheibe

Aufgabe

- Den Turm auf den rechten Stab bringen

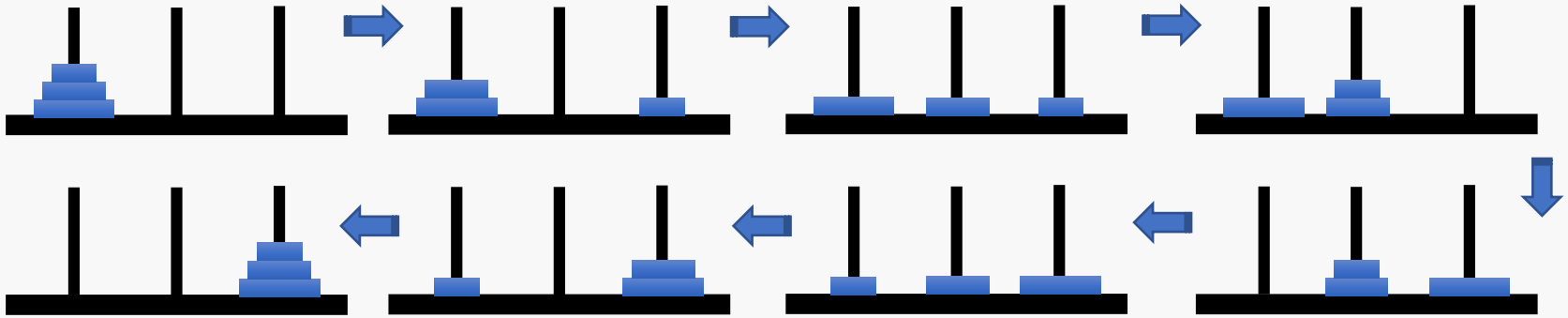
Einschränkungen

- Nur die oberste Scheibe darf bewegt werden
- Es darf nicht eine Scheibe auf eine kleinere Scheibe gelegt werden



Türme von Hanoi – Ablauf

n = 3



n=3 n=3

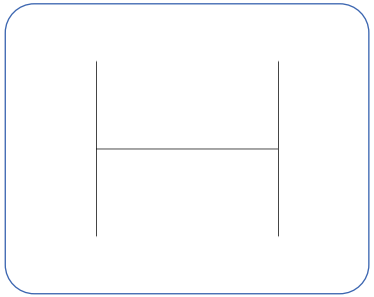
n = 4 (verkürzt)

Türme von Hanoi – Implementierung

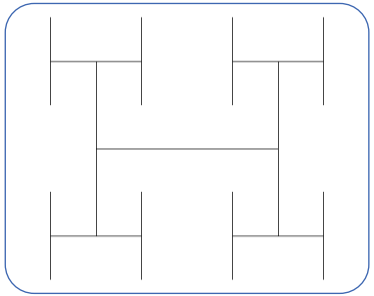
```java
public class TowersOfHanoi {

    private static void solveHanoi(int n, String a, String b, String c) {
        if (n > 0) {
            solveHanoi(n - 1, a, c, b);
            System.out.println(a + " --> " + c);
            solveHanoi(n - 1, b, a, c);
        }
    }

    public static void main(String[] args) {
        solveHanoi(3, "Tower 1", "Tower 2", "Tower 3");
    }
}
```

```
Tower 1 --> Tower 3
Tower 1 --> Tower 2
Tower 3 --> Tower 2
Tower 1 --> Tower 3
Tower 2 --> Tower 1
Tower 2 --> Tower 3
Tower 1 --> Tower 3
```

Verzweigte Rekursion und Grafik (H-Baum)



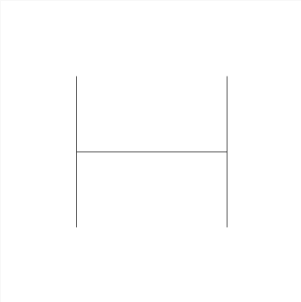
Ein H mit drei Linien zeichnen

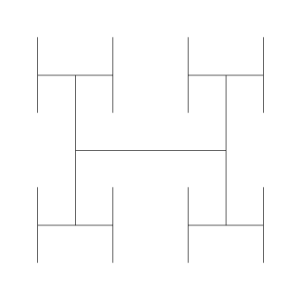


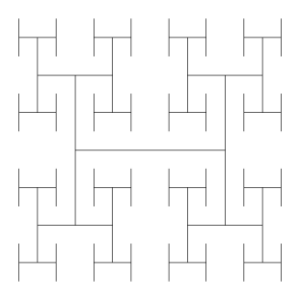
Rekursionsschritt

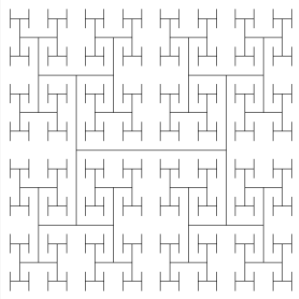
- Vier H-Bäume für n – 1 zeichnen
- Jeweils mit den Spitzen des H-Baums für n verbinden
- Jeder H-Baum für n – 1 ist nur halb so groß wie der H-Baum für n (Basisfall, d. h. nichts zeichnen, wenn n <= 0)

H-Baum der Ordnung 1, 2, 3, 4









Htree.java in TUWEL

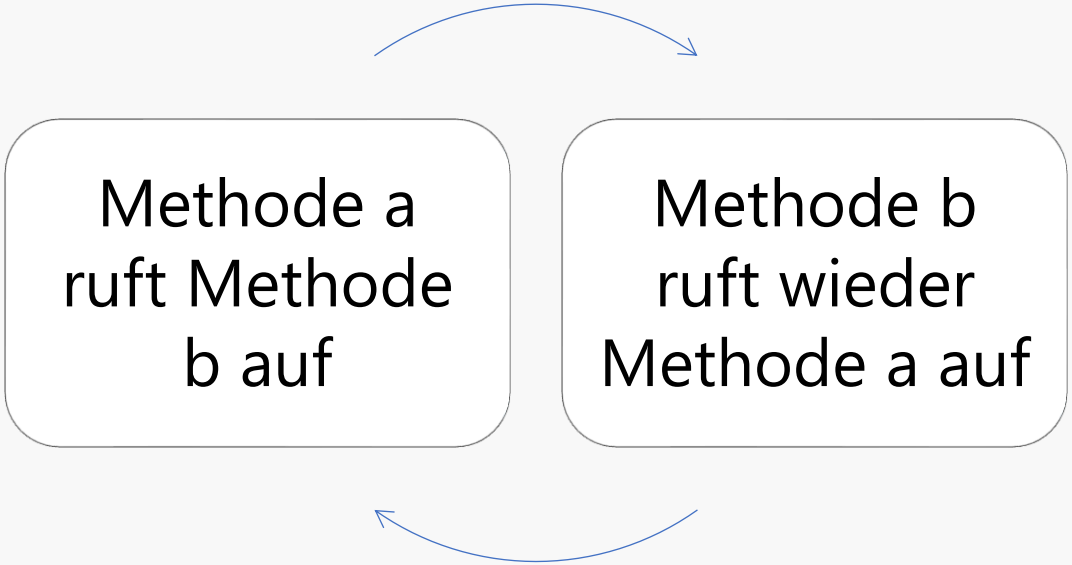
Wechselseitige Rekursion

Der rekursive Aufruf erfolgt indirekt

- Es können auch mehr als zwei Methoden an der Rekursion beteiligt sein
- Wird auch als indirekte Rekursion oder alternierende Rekursion bezeichnet

Hinweis
Beim Lösen eines Problems *P* werden rekursive Lösungen für Subprobleme verwendet, die selbst nicht kleinere Instanzen von *P* sind

Wechselseitige Rekursion – Beispiel



Beispiel (gerade/ungerade überprüfen)

```java
public class EvenOddTest {

    private static boolean even(int n) {
        if (n == 0)  {
            return true;
        }
        return odd(n - 1);
    }

    private static boolean odd(int n) {
        if (n == 0) {
            return false;
        }
        return even(n - 1);
    }

    public static void main(String[] args) {
        System.out.println(even(1));
        System.out.println(even(2));
        System.out.println(odd(3));
        System.out.println(odd(4));
    }

}
```

```
false
true
true
false
```

Verschachtelte Rekursion

Argumente bei einem rekursiven Aufruf sind selbst wieder Rekursionsaufrufe

Vorhergehender Aufruf muss abgeschlossen sein, bevor ein weiterer gestartet werden kann

Beispiel (Ackermann-Funktion)

```java
public class AckermannTest {

    // n >= 0, m >= 0
    private static long ackermann(long n, long m) {
        if (n == 0) {
            return m + 1;
        } else if (m == 0) {
            return ackermann(n - 1, 1);
        } else {
            return ackermann(n - 1, ackermann(n, m - 1));
        }
    }

    public static void main(String[] args) {
        System.out.println(ackermann(1, 0));
        System.out.println(ackermann(2, 1));
        System.out.println(ackermann(2, 2));
        System.out.println(ackermann(3, 3));
    }

}
```

```
2
5
7
61
```

Ackermann-Funktion – Verwendung

Benchmarks

- Anzahl der Aufrufe wächst extrem schnell

Analyse von Algorithmen

Wichtige Funktion in der theoretischen Informatik